

Back To Basics

Memory Alignment

SARTHAK SEHGAL

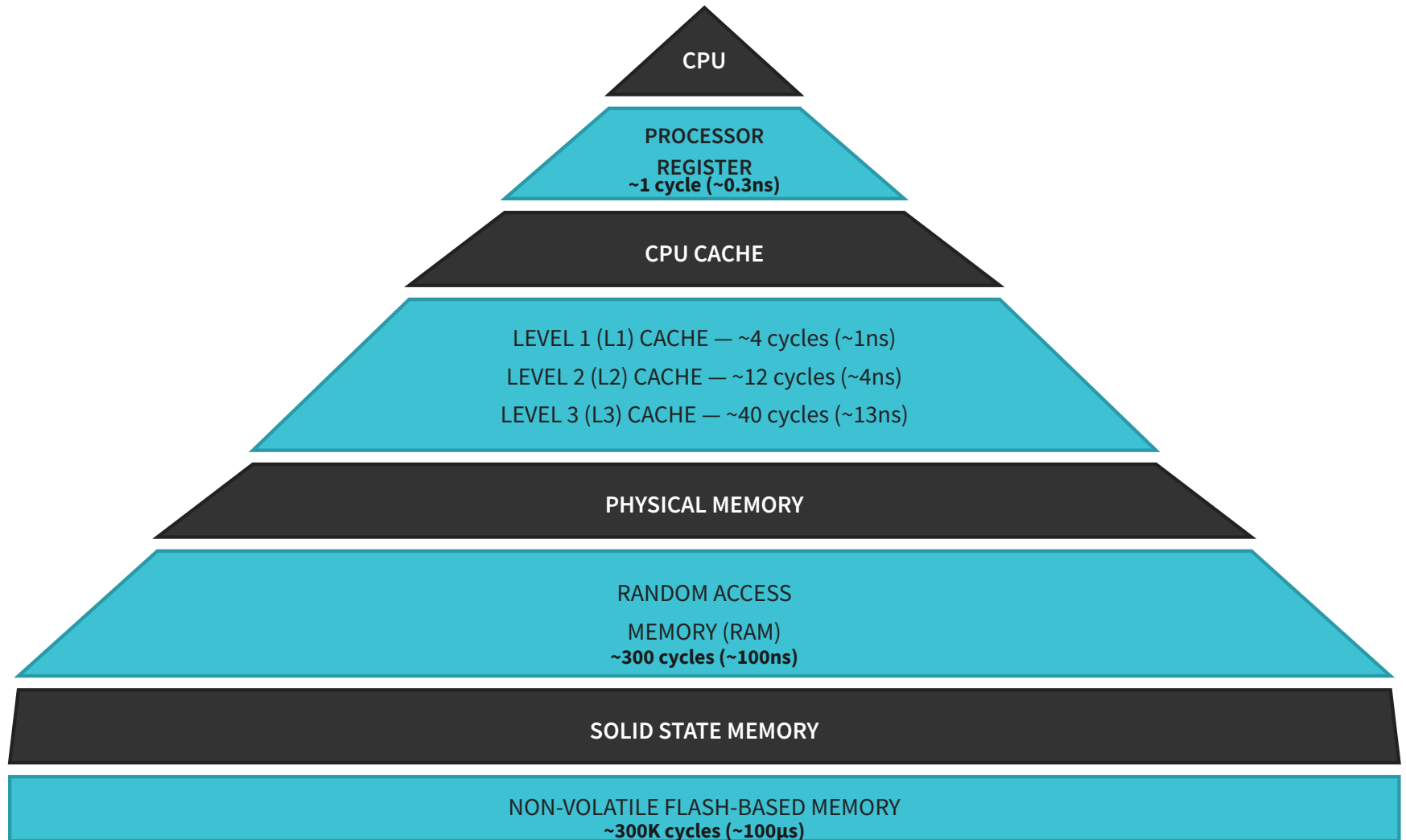


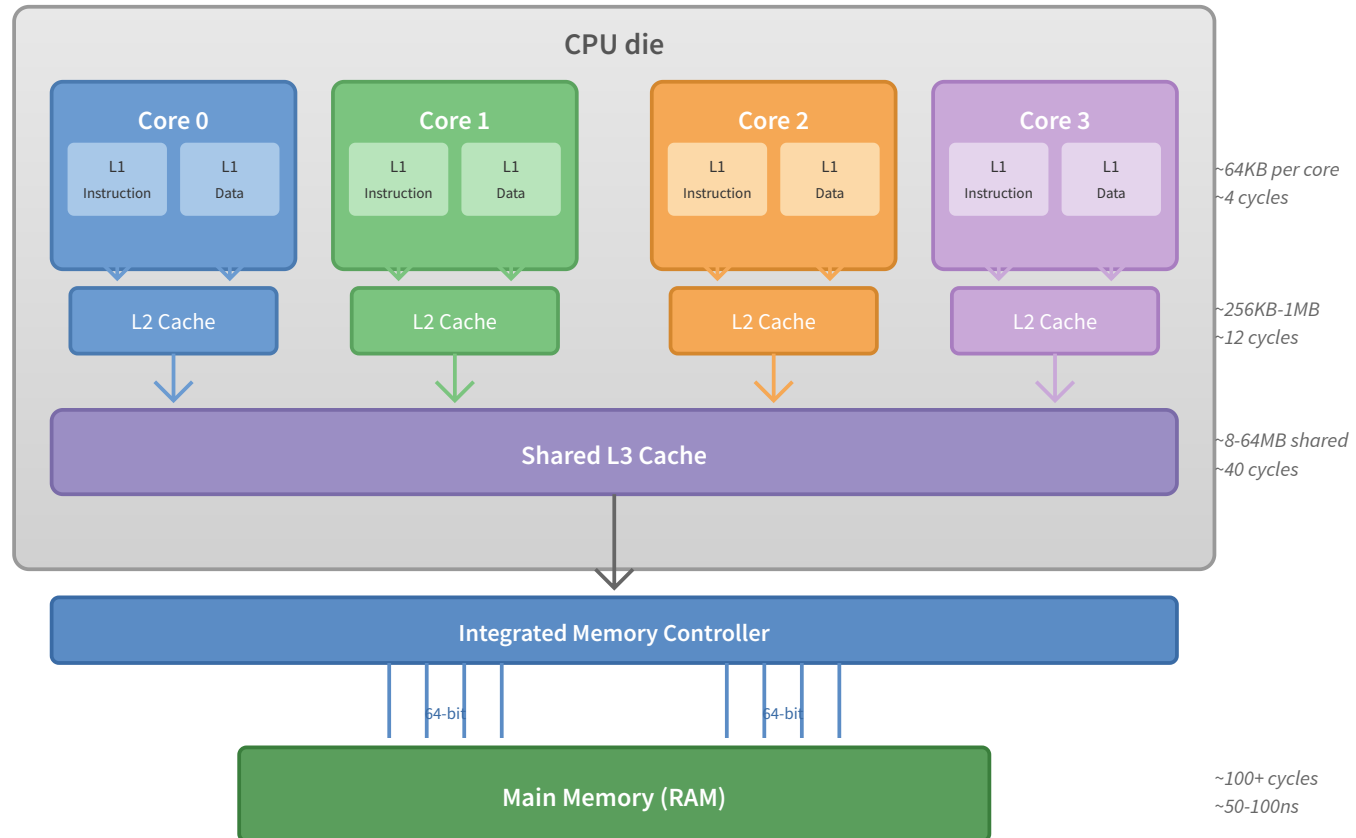
OVERVIEW

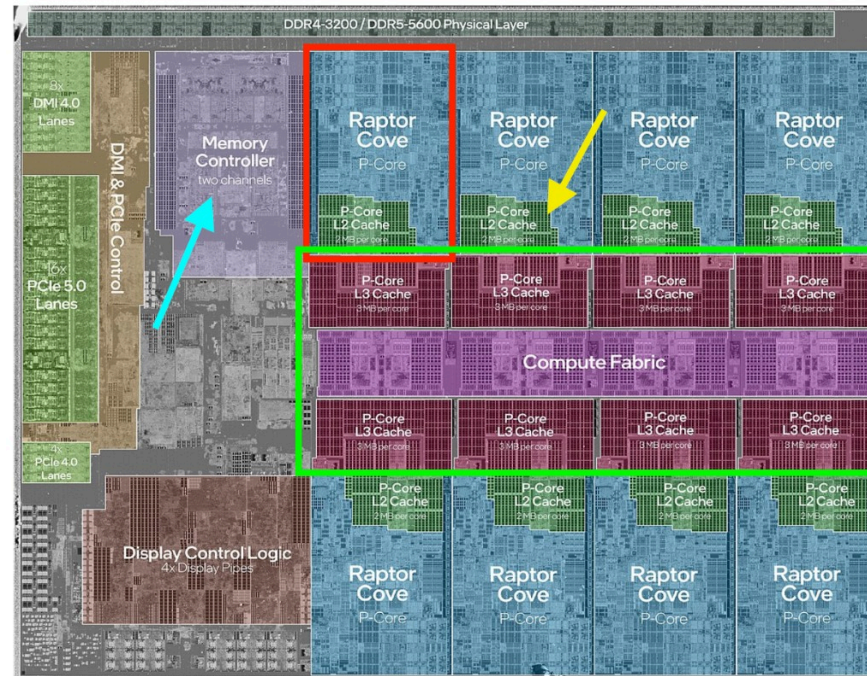
- CPU Memory Access & Alignment
- Compiler Memory Layout, Padding, and Packing
- Controlling Alignment & Handling Misaligned Access

Understanding How Modern CPUs Access Memory

CPU Memory Hierarchy



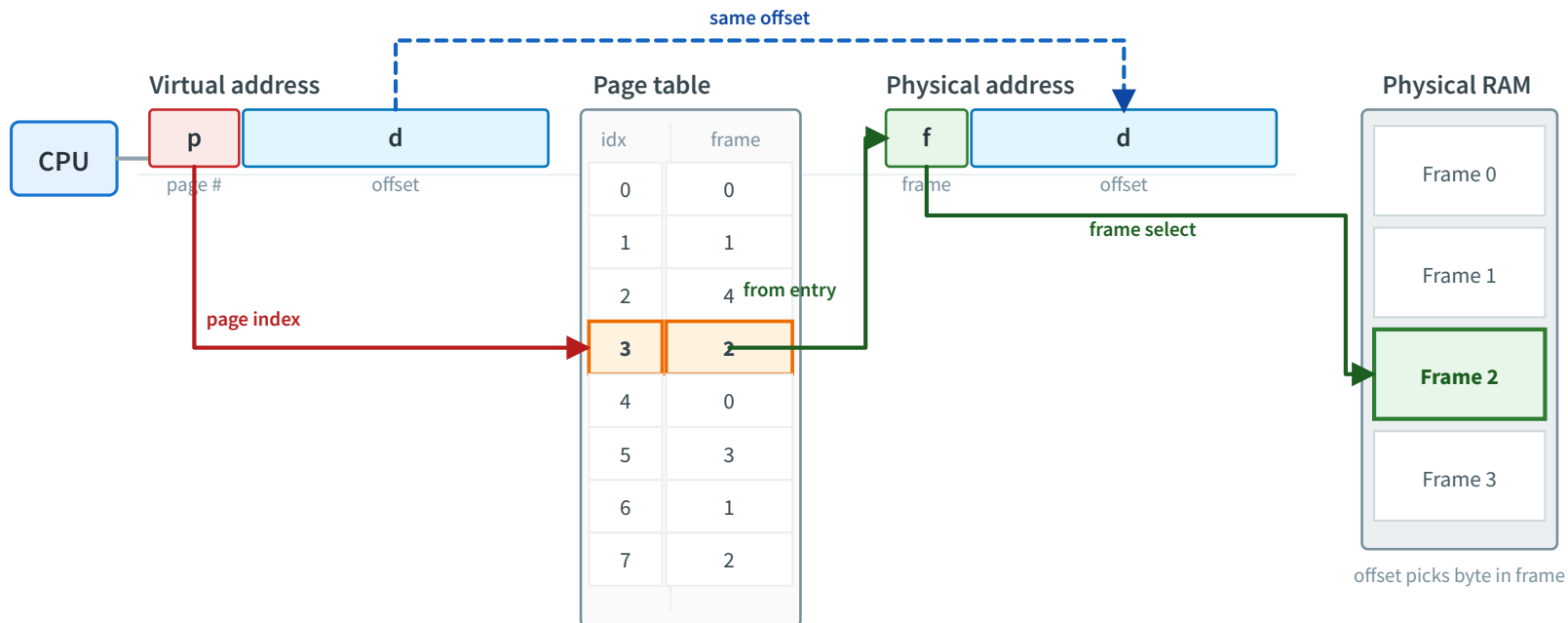




Intel Core i9-13900K

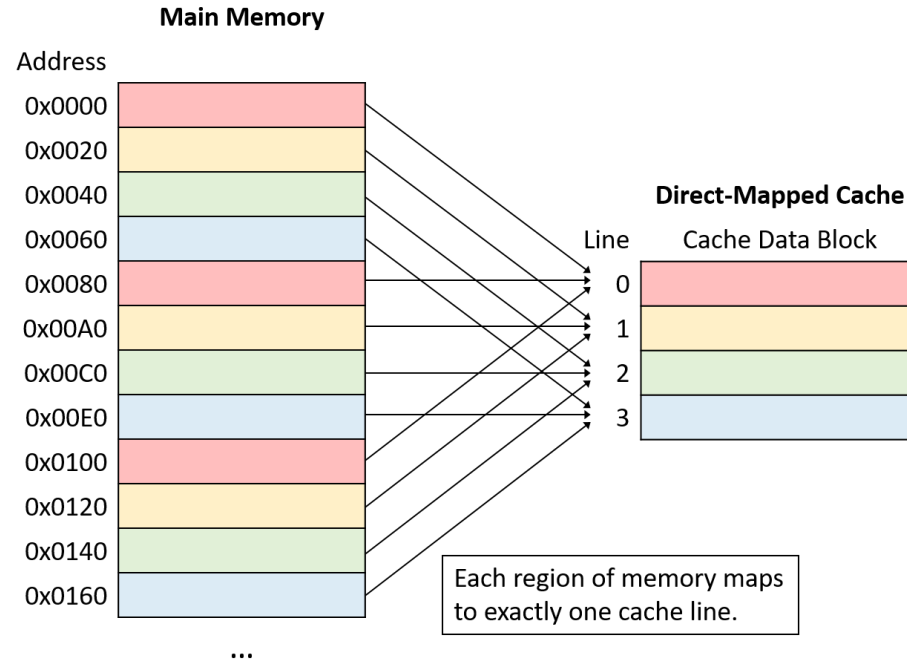
Main memory access

Physical memory is divided into fixed-size frames. Each process has its own virtual address space; the Memory Management Unit (MMU) translates virtual addresses to physical ones using page tables that map pages to frames.



Cache Lines

Caches transfer data in fixed-size **cache lines**, not arbitrary bytes. Each row holds one line and metadata (tag, valid, ...) pointing at a block in RAM.



Why Cache Works: Locality

TEMPORAL LOCALITY

"If you accessed it once, you'll likely access it again soon"

Example: loop counter, frequently called function

SPATIAL LOCALITY

"If you accessed address X, you'll likely access X+1, X+2..."

Example: array iteration, struct fields

Accessing Memory in a Program

```
1 int x = array[5];
```

"I need 4 bytes at address 0x1014"

What CPU does:

1. Check: is cache present and valid?
2. If NO → fetch 64 bytes from RAM
3. Store entire line in cache
4. Return the 4 bytes you asked for

Cache Line (addresses 0x1000 - 0x103F)



✓ Entire 64-byte line now in cache — next nearby access is fast!

Word Boundaries and Memory Alignment

On a 64-bit system: **word size = 8 bytes**



Modern CPUs read memory word-by-word — one word (8 bytes) per memory access

Memory Alignment = placement or alignment of data in the memory. Data types are required to be placed at addresses that are multiples of type's alignment:

- `int32_t` (4 bytes) → aligned at addresses 0, 4, 8, 12...
- `int64_t` (8 bytes) → aligned at addresses 0, 8, 16, 24...
- `char` (1 byte) → aligned at any address

Aligned vs Misaligned Memory Access

Reading a 4-byte `int` at address 4:

0	1	2	3	4	5	6	7
---	---	---	---	---	---	---	---

✓ Single memory fetch

✓ Efficient hardware operation

Reading a 4-byte `int` at address 6:

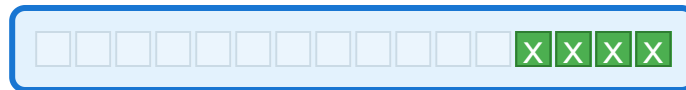
0	1	2	3	4	5	6	7
8	9	10	11	12	13	14	15

✗ Spans two word boundaries

✗ Requires two memory fetches + combine

Caching and Misaligned Access

Reading a 4-byte `int` at offset 60 (same cache line):



✓ ONE cache line fetch

Reading a 4-byte `int` at offset 62 (spans cache lines):



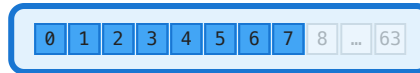
✗ TWO cache line fetches required

✗ CPU must combine bytes from both lines

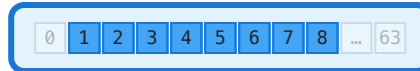
Benchmarking Memory Access

Copy a `uint64_t` (8 bytes) at different offsets within a 64-byte cache line:

1. Aligned — offset 0



2. Misaligned (same line) — offset 1

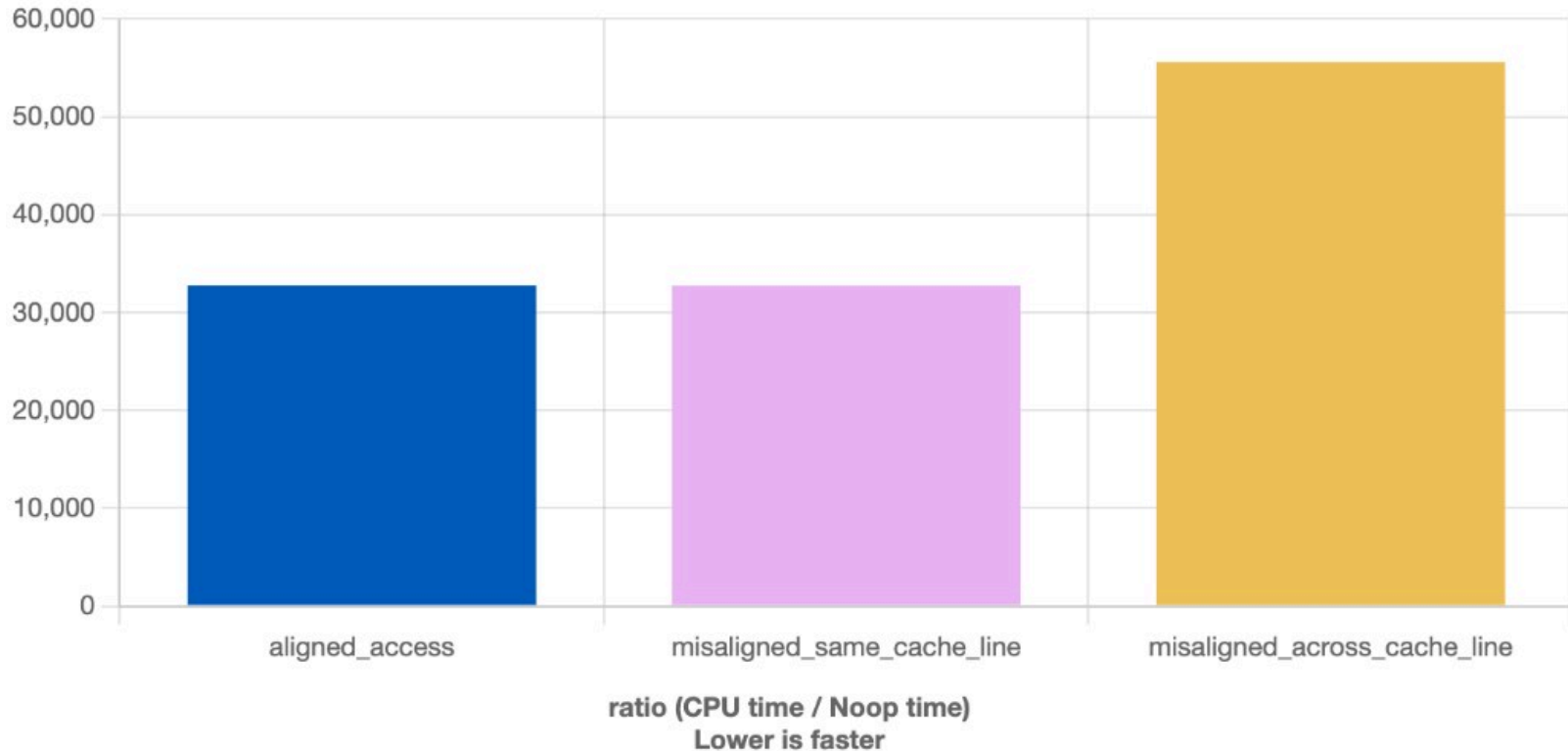


3. Misaligned (spans cache line) — offset 60



[Compiler Explorer](#) →

The Results



- ✓ `aligned_access` $\approx$ `misaligned_same_cache_line`
- ✗ `misaligned_across_cache_line` $\approx$ **1.7× slower**

Conclusion

- Modern CPUs handle misaligned access fast — just not *optimally*.
- It's not misalignment that hurts the most, it's **crossing a cache line**.
- Dereferencing a misaligned pointer is **undefined behavior** — that's why we use `memcpy` instead of `reinterpret_cast`.

Compiler Memory Layout of Structures

Alignment & Padding

```
1 struct Foo {  
2     char a;    // 1 byte  
3     int b;     // 4 bytes  
4     char c;    // 1 byte  
5 };  
6  
7 // What is sizeof(Foo)?
```

[Compiler Explorer →](#)

[Cplusplus →](#)

Power of Member Reordering

Before

```
1 struct Bad {
2     char a;    // 1
3     // 3 pad
4     int b;     // 4
5     char c;    // 1
6     // 3 pad
7 };
8 // sizeof = 12
```

After

```
1 struct Good {
2     int b;     // 4
3     char a;    // 1
4     char c;    // 1
5     // 2 pad
6 };
7 // sizeof = 8
```

33% smaller just by reordering!

Inspecting Layout with **offsetof**

offsetof(type, member) — returns byte offset of a member within a struct

```
1  #include <stddef>
2
3  struct Foo {
4      char a;      // 1 byte
5      int b;       // 4 bytes
6      char c;      // 1 byte
7  };
8
9  static_assert(offsetof(Foo, a) == 0);
10 static_assert(offsetof(Foo, b) == 4); // 3 bytes padding after 'a'
11 static_assert(offsetof(Foo, c) == 8);
12 static_assert(sizeof(Foo) == 12);    // 3 bytes tail padding
```

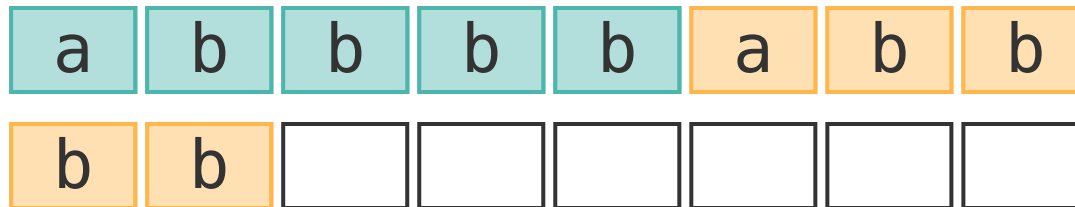
Only guaranteed for standard-layout types

Why Struct Alignment = Largest Member?

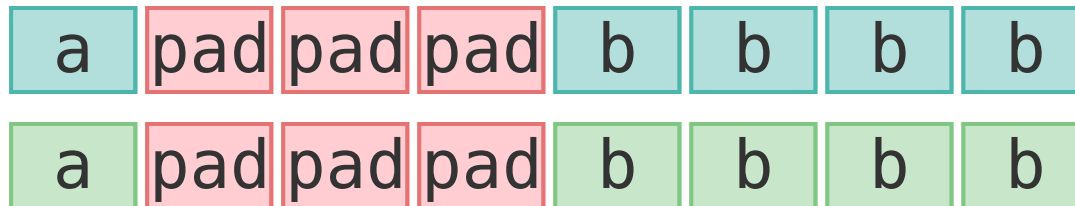
Consider an array of structs with wrong alignment (1-byte):

```
struct Foo { char a; int b; }; // If alignof(Foo) were 1...
```

Foo arr[2] — with 1-byte alignment (WRONG)



Foo arr[2] — with 4-byte alignment (CORRECT)



✓ arr[1].b starts at offset 12 — aligned!

Alignment Rules

- Every type has an alignment: `alignof(T)` — always a power of 2.
- Fundamental types are typically aligned to their size (natural alignment).
- Structs and classes are aligned to the largest alignment of any member; size is padded to a multiple of that alignment.
- Pointers: a `T*` is assumed by the compiler to be aligned to `alignof(T)` — violating this is Undefined Behavior.
- The compiler guarantees variables are placed at properly aligned addresses.
- `std::max_align_t`: the largest/strictest fundamental alignment.

Misaligned Pointer = Undefined Behavior

```
1 char buf[4] = {'1', '2', '3', '4'}; // alignment of char (1 byte)
2 static_assert(sizeof(buf) == sizeof(int));
3
4 int* p = reinterpret_cast<int*>(buf); // Undefined behavior – mis
5
6 *p = 42;
```

- The compiler *assumes* `p` is 4-byte aligned and may emit aligned-only instructions.
- Safe alternative: `std::memcpy`

Safe Access with memcpy

```
1 alignas(int) char buf[4] = {'1', '2', '3', '4'};
2 static_assert(sizeof(buf) == sizeof(int));
3
4 int* p = std::start_lifetime_as<int>(buf); // C++23
5 std::memcpy(&p, buf, sizeof(p)); // Safe – no alignment assumption
```

C++23: `std::start_lifetime_as<T>` — starts object lifetime without copying

Saving Space with Packing

Packed Structures

Remove padding for compact memory layout

```
1 // GCC/Clang
2 struct __attribute__((packed)) PackedData {
3     char a;
4     int b;
5     char c;
6 }; // sizeof = 6 (no padding!)
7
8 // MSVC
9 #pragma pack(push, 1)
10 struct PackedData {
11     char a;
12     int b;
13     char c;
14 };
15 #pragma pack(pop)
```

When to Use Packed Structs

- **Network protocols** - match wire format exactly
- **File formats** - binary file headers
- **Hardware registers** - memory-mapped I/O
- **Memory-constrained systems** - embedded devices
- **Large arrays** - when memory savings matter

Packed Struct: Network Packet

```
1 // Network packet header – must match wire format exactly
2 struct __attribute__((packed)) PacketHeader {
3     uint8_t  version;           // 1 byte
4     uint32_t sequence;          // 4 bytes
5     uint8_t  type;              // 1 byte
6     uint64_t timestamp;         // 8 bytes
7 }; // 14 bytes packed, would be 24 unpacked
8
9 static_assert(sizeof(PacketHeader) == 14,
10               "Header size must match protocol spec");
```

Dangers of Packed Structs

```
1 struct __attribute__((packed)) Packed {
2     char c;
3     int value; // Misaligned!
4 };
5
6 void dangerous(Packed* p) {
7     int* ptr = &p->value; // Creating misaligned pointer
8     *ptr = 42;             // Undefined behavior!
9 }
```

Taking address of packed member = UB

Bit-fields Packing

```
struct Flags {  
    bool is_active;  
    bool is_visible;  
    bool is_dirty;  
}; // sizeof = 3 bytes
```

Bit-fields Packing

Pack multiple booleans into a single byte

```
struct Flags {  
    bool is_active : 1;  
    bool is_visible : 1;  
    bool is_dirty : 1;  
}; // sizeof = 1 byte
```

Bit-fields: Real-World Example

```
struct NotSendingReason {  
    bool tradingDisabled : 1 = true;  
    bool orderThrottle    : 1 = true;  
    bool moveTooLarge      : 1 = true;  
    bool moveTooSmall      : 1 = true;  
    bool reserved          : 4 = false;  
  
    constexpr operator bool() const {  
        return std::bit_cast<std::uint8_t>(*this) != 0u;  
    }  
};
```

- Compact representation of multiple rejection reasons
- Implicit conversion to `bool` — check if any reason is set
- `std::bit_cast` (C++20) for type-safe reinterpretation

std::vector<bool> — A Cautionary Tale

`std::vector<bool>` is a specialization that packs bits

[Compiler Explorer →](#)

```
std::vector<bool> vb = {true, false, true};
bool* ptr = &vb[0];    // Error! Can't take address of a bit
auto& ref = vb[0];     // Not a bool& – it's a proxy object

std::vector<int> vi = {1, 2, 3};
int* p = &vi[0];       // Fine
int& r = vi[0];        // Fine
```

Why it's problematic:

- `operator[]` returns a proxy, not a reference
- Breaks generic code expecting `T&`

Controlling Alignment

alignas and alignof (C++11)

- `alignof(T)` — query the alignment requirement of a type
- `alignas(N)` — request N-byte alignment for a variable or type

```
1 // alignof – inspect
2 static_assert(alignof(int) == 4);
3 static_assert(alignof(double) == 8);
4
5 // alignas – control
6 alignas(64) int cache_line_aligned_data[16];
7
8 struct alignas(16) Vec4 {
9     float x, y, z, w;    // 16 bytes → one SSE vector
10 };
11
12 static_assert(alignof(Vec4) == 16);
```

alignas Rules

- Alignment must be a **power of 2**
- Cannot be **weaker** than natural alignment
- Can use `alignas(alignof(T))` to match another type

```
1 alignas(1) int x;           // Error! Can't weaken alignment
2 alignas(8) int y;           // OK: stricter than natural (4)
3 alignas(7) int z;           // Error! Not a power of 2
4
5 alignas(alignof(double)) int d; // OK: align int like double
```

Aligning Raw Storage

```
1 // Stack buffer for 10 ints – but std::byte has 1-byte alignment!
2 std::byte buffer[10 * sizeof(int)];
3
4 int* arr = reinterpret_cast<int*>(buffer);
5 arr[0] = 42; // UB – buffer may not be 4-byte aligned
```

Use `alignas` to fix:

```
1 // Properly aligned storage for 10 ints
2 alignas(int) std::byte buffer[10 * sizeof(int)];
3
4 int* arr = reinterpret_cast<int*>(buffer);
5 arr[0] = 42; // Safe – buffer is 4-byte aligned
```

Practical Alignment: Memory Pool in C++

```
1  template <typename T, std::size_t N>
2  class MemoryPool {
3      alignas(T) std::byte storage[sizeof(T) * N];
4      std::size_t used = 0;
5
6  public:
7      template <typename... Args>
8      T* allocate(Args&&... args) {
9          if (used >= N) return nullptr;
10         void* slot = storage + sizeof(T) * used;
11         ++used;
12         return std::construct_at(
13             reinterpret_cast<T*>(slot), std::forward<Args>(args)...);
14     }
15 };
```

`alignas(T)` ensures storage is properly aligned for `T`

Aligning memory at runtime

`std::align ( <memory> )`

```
1 void* align(std::size_t alignment,  
2           std::size_t size,  
3           void*& ptr,  
4           std::size_t& space);
```

Given a pointer `ptr` to a buffer of size `space`, returns a pointer aligned to `alignment` for `size` bytes and decreases `space` by the number of bytes used for alignment.

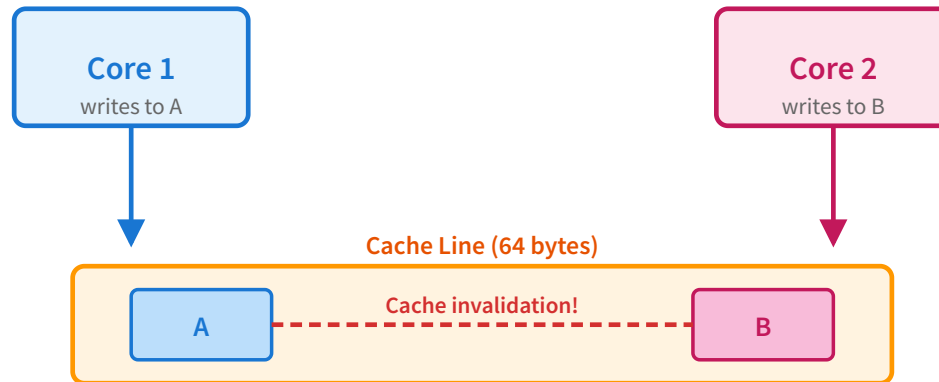
`std::align` example

```
1  std::byte buffer[64];
2  void* ptr = buffer + 1;
3  std::size_t space = sizeof(buffer)-1;
4
5  std::cout << "Space before = " << space << '\n';
6  std::cout << "Address = " << std::addressof(ptr) << '\n';
7
8  // Align to 8 bytes; reserve room for one int
9  if (void* aligned = std::align(8, sizeof(int), ptr, space))
10 {
11     std::cout << "Space after = " << space << '\n';
12     std::cout << "Address = " << std::addressof(aligned) << '\n';
13     auto* value = static_cast<int*>(aligned);
14     *value = 42;
15     ptr = static_cast<std::byte*>(ptr) + sizeof(int);
16     space -= sizeof(int);
17 }
```

[Compiler Explorer →](#)

False Sharing

When threads on different cores modify variables that share a cache line



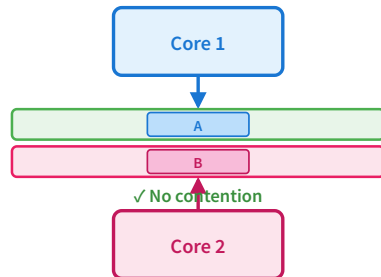
Core 1 writes A → invalidates Core 2's cache line

Core 2 writes B → invalidates Core 1's cache line

Constant cache line bouncing = poor performance

Fixing False Sharing with Alignment

Put each variable on its own cache line:



```
1 struct Counters {  
2     alignas(64) int counter1;  
3     alignas(64) int counter2;  
4 };
```

C++17: `std::hardware_destructive_interference_size`
— minimum offset to avoid false sharing

Summary

- CPUs access memory most efficiently at aligned addresses
- Crossing cache line boundaries hurts performance
- Compilers add padding to maintain alignment; struct alignment = largest member
- Use `alignas`, `alignof`, `sizeof`, `std::align` to inspect and control
- Packed structs and bit-fields save space but require careful handling
- Pointers to packed members escape the "packed context" — use `memcpy`

Questions?